Constructors: where objects begin

Today

Welcome to your next lesson: Constructors: where objects begin.

I'm getting your personalized lesson ready—built around what you've learned, what's next, and the best way to help it stick.

Keep it up, every step counts.

Imagine every new pet that arrives at a shelter 🐶🐱. You don’t want a totally “empty” record — you at least want a basic starting record like: name unknown, species pet, age 0.

Constructors give your objects that automatic starting state.

In Python, that “startup” is done in a special method called __init__ (we say “dunder init”). Python runs it automatically when you do something like luna = Pet().

Before we go deeper:

Have you already seen __init__ before, even briefly, or is this your first time really looking at it?

I’ve seen it before

Nice, that helps — we can move a bit faster then ⚙️

Here’s the one new idea I want to lock in now:

The constructor in Python is the __init__ method.

It runs automatically right after you create an object, and

it sets up that object’s starting attributes.

Very simple example in our pets world:

Python
```
class Pet:
    def __init__(self):
        self.name = "Unknown"
        self.species = "pet"

luna = Pet()
print(luna.name)      # Unknown
print(luna.species)   # pet
```
Step-by-step flow is:

Pet() → Python makes a new object → __init__ runs → self.name and self.species get created on that object ✅

Can you describe that flow back to me in your own words, in 1–2 short sentences?

Don’t worry about perfect wording — I just want to see how you’re seeing it.

when you create an object from the class

Exactly, you’re on the right track 👍

Let me build on your sentence, step by step:

You create an object: luna = Pet()
Python makes a fresh Pet object in memory
Python automatically calls __init__ on that new object
Inside __init__, lines like self.name = "Unknown" create the starting attributes on that object

So you only write Pet(). Python takes care of calling __init__ for you.

Want to see that flow as a little visual diagram next? 👀

hokdtypo

Got you 😄 I'll take that as "hold on / typo".

Let's keep going, one clear piece at a time.

Now I want to zoom in on what __init__ actually does inside the object.

In this code:

Python

```
class Pet:
    def __init__(self):
        self.name = "Unknown"
        self.species = "pet"
```

Line by line inside __init__:

self.name = "Unknown" → creates an attribute called name on this specific pet and gives it "Unknown".
self.species = "pet" → creates an attribute called species on this specific pet and gives it "pet".

So __init__ is like the "starter pack" builder for each new pet.

Does this part make sense so far:

that self.name = ... and self.species = ... inside __init__ are how each new object gets its default attributes?

when you create an object from the class pet, python creates the object and automatically runs _init which initializes self.name and self.species

Perfectly said 💯

One tiny detail: the method name is __init__ (two underscores before and after), not __init_. But your understanding is spot on:

create luna = Pet()
Python makes the object

```
Python automatically runs __init__
that code sets self.name and self.species on the new pet
Next step: I’d like you to use this.


In the code editor, try this:


Define a Pet class with an __init__(self) that sets:
self.name = "Unknown"
self.species = "pet"
self.age = 0
Then create two pets, like pet1 = Pet() and pet2 = Pet().
print their name, species, and age.

Run it, then Submit code so I can see it, and I’ll give you
feedback 🎯

Python
class Pet:
    def __init__(self):
        self.name = "Unknown"
        self.species = "pet"
        self.age = 0
pet1 = Pet()
pet2 = Pet()
print(f'Pet1 is {pet1.name} whom is a {pet1.species} and is
{pet1.age} years old')
print(f'Pet2 is {pet2.name} whom is a {pet2.species} and is
{pet2.age} years old')

This is excellent, Ron 🎯


Your __init__ is correct.
All default attributes are set on self.
Both pet1 and pet2 start with the same defaults, and your
prints prove it.

So we’ve now used a constructor to give objects a predictable
starting state. ✅

One more key idea, very briefly:



You do not call __init__ yourself.
```

You only write Pet() — Python calls __init__ for you automatically.

Make sense that __init__ is not something you ever write like pet1.__init__() in normal code?

If yes, I'll give you a quick check-question using a little multiple choice.

yes i almost menioed that _ and -- denote uncallable methods do they not

What do you think? 👇

When luna = Pet() runs, who gives Luna her starting attributes?

A.
The print() statement

B.
The __init__ method

C.
The variable name luna

D.
The describe() method

Nice catch bringing that up, Ron 👀

Quick clarification:

Methods with double underscores like __init__ are called "dunder" special methods. They are callable—Python just calls them for you in special situations (like __init__ when you do Pet()), so you rarely call them directly.

A single leading underscore (like _helper) is just a convention meaning “internal use”, not “uncallable”. It’s still a normal, callable method.

So __init__ absolutely does run; it’s just Python that calls it automatically, not you with dot notation.

You also nailed the quiz: the __init__ method is exactly what gives luna her starting attributes when luna = Pet() runs 🎯

Want to do one more small practice class with a constructor (maybe not pets this time), or are you ready to wrap this lesson and move on?

Let's practice another class

Love it 💪 Let’s do one more, same idea but a different world.

We’ll make a simple Car class that starts with default values, just like your Pet.

Step by step, here’s what I want you to do in the code editor:

Define a class Car.
Add an __init__(self) that sets default attributes on self, for example:
self.make = "Unknown"
self.model = "Unknown"
self.year = 2000
Create two cars, like car1 = Car() and car2 = Car().
print each car’s attributes in a clear sentence (like you did with pets).

Then hit Submit code so I can see it 🚗

I’ll check that the constructor is doing all the setup automatically.

Python
```
class Car:
    def __init__(self):
        self.make = "Unknown"
        self.model = "Unknown"
```

```
        self.year = 2000
car1 = Car()
car2 = Car()
print(f'car1 is a {car1.year}, make {car1.make}, model {car1.model} automobile')
print(f'car2 is a {car2.year}, make {car2.make}, model {car2.model} automobile')

Take your time with the question below 👇

When luna = Pet() runs, who gives Luna her starting attributes?


A.
The print() statement


B.
The __init__ method


C.
The variable name luna


D.
The describe() method

This is great, Ron 🎯


Your Car class uses __init__ exactly how I wanted:


__init__ runs automatically when you do Car()
It sets self.make, self.model, and self.year so every new car starts with a predictable state
Both car1 and car2 show those defaults correctly in your printout

And your quiz answer is spot on: when luna = Pet() runs, it’s the __init__ method that gives Luna her starting attributes.

print() only shows the attributes. The variable name luna just points to the object. The work of creating and initializing the attributes happens inside __init__.
```

Quick check to close this out:

If you read this line in someone's code:

Python
robot = Robot()
In your own words: what do you expect is happening with __init__ at that moment?

its is initiazizing the state of the class attributes (vatables) to a known default state

Beautifully put ✅

Small wording tweak: it initializes the state of the object's instance attributes to a known default state. But the idea you have is exactly right:

robot = Robot() → Python creates a new Robot object
Then Robot.__init__ runs automatically
Inside __init__, it sets up that robot's starting attributes (default values) so the object isn't "empty"

You've hit all the key points for this lesson:

You understand what __init__ is and its syntax
You know it runs automatically when you create an object
You wrote multiple classes that use __init__ to assign default attributes on self 🎯

Want to wrap up this constructors lesson and move on to the next one, or keep playing with one more custom class for extra practice?

Let's wrap up and move on

Awesome work today, Ron 🎯

Here's what you're taking with you from this lesson:

You know that the constructor in Python is the __init__ method.
You understand that __init__ runs automatically right after you create an object with Thing().
You wrote your own classes (Pet, Car) where __init__ gives each new object default attributes through self.

In short: you now know where objects begin and how to give them a clean, predictable starting state.

As you go into the next lesson, keep this mental picture:

MyClass() → Python creates an object → __init__ runs → default attributes appear on self ✅

Whenever you're ready in the Learn tab, you can jump into the next step. I'll be here to keep building on this with you.


```
class Pet:
    def __init__(self):
        self.name = "Unknown"
        self.species = "pet"
        self.age = 0
pet1 = Pet()
pet2 = Pet()
print(f'Pet1 is {pet1.name} whom is a {pet1.species} and is {pet1.age} years old')
print(f'Pet2 is {pet2.name} whom is a {pet2.species} and is {pet2.age} years old')

class Car:
    def __init__(self):
        self.make = "Unknown"
        self.model = "Unknown"
        self.year = 2000
car1 = Car()
car2 = Car()
print(f'car1 is a {car1.year}, make {car1.make}, model {car1.model} automobile')
print(f'car2 is a {car2.year}, make {car2.make}, model {car2.model} automobile')
```

352
353